do_while program contd..

```
DoWhileDemo5.java
-----------------------
void main()
{
     int i = 1; //i = 5

    do
     {
     IO.println(i);  //1  2  3  4
    }
     while (i++ <= 3);  //4 <=3

}
```

```
DoWhileDemo6.java
---------------------
void main()
{
     int i = 1; //6

    do;
      while (i++ < 5); //5 < 5

    IO.println(i);
}
```

```
DoWhileDemo7.java
-----------------------
void main()
{
   int i = 1; //i=5

   do
     {
     IO.println(i); //1   2   3  4
    }
   while ((i = i + 1) < 5);  //5 < 5

}
```

```
DoWhileDemo8.java
------------------------
void main()
{
 int i = 1;  //i = 7

 do
 {
  IO.print(i++ + " ");  //1  3  5
 }
 while (i++ < 5);  //6 < 5

 IO.println("\ni = " + i);

}
```

```
++x;  // use new value

x++; //use old value and increment the value of
         x by 1 in the memory

--x; //use new value

x--; //use old value and decrement the value of
         x by 1 in the memory
```

2) while loop:

It is an **entry-controlled loop** because first, it will verify the **condition**, If Condition is **true** then only body will be executed.
It is preferable when number of iterations are **not fixed**.

```
Syntax:
while(condition)
{
  // statements
}
```

```
WhileDemo1.java
--------------------
//Print the value from 0 to -10 using while loop
void main()
{
     int x = 0;

   while(x >=-10)
     {
          IO.print(x+"\t");
          x--;
     }

}
```

```
WhileDemo2.java
--------------------
void main()
{
     int i = 1; //6
     while(i++ < 5); //5 < 5
     IO.println(i);


}
```

```
WhileDemo3.java
-------------------
void main()
{
     int i = 1;  //i=7

     while(i++ < ++i)  //5 <  7
     {
          IO.println(i); //3  5  7

          if(i > 5)
               break; //When the if condition become true then break will execute
     }
}
```

```
WhileDemo4.java
--------------------
void main()
{
     while(false)
     {
          IO.println("Hello");  //Unreachable code
     }

     IO.println("Hi");
}
```

The body of while loop will not be executed
so it becomes unreachable statement

```
WhileDemo5.java
-------------------
void main()
{
     boolean b = false;
     while(b)
     {
          IO.println("Hello");
     }

     IO.println("Hi");      //Output is Hi

}
```

```
WhileDemo6.java
--------------------
void main()
{
     final int x = 10;
     final int y = 20;

     while(x>y)         //due to final  variable body of while will not be executed so, unreachable
     {
          IO.println("x is greater than y");
     }

     IO.println("Hello learner!!!");
}
```

```
WhileDemo7.java
-------------------
void main()
{
     int i = 1;  //i=7

     while(i <= 5)  //5 <= 5
     {
          IO.print(i++ + "\t"); //1  3  5
          i++;
     }

     IO.println("Final value of i "+i);
}
```

3) for loop:

It is also **entry-controlled loop,** it is used to write concise code that means everything initialization, condition, and increment/decrement written in a single line.
It is preferable when number of iterations are **fixed**.

```
Syntax:
for (initialization; condition; increment/decrement)
{
   // statements
}
```

```
ForLoopDemo1.java
----------------------
void main()
{
     for(int i=1; i<=10; i++)
     {
          IO.println("Welcome to Java");
     }

}
```

```
ForLoopDemo2.java
----------------------
//Print all the even numbers from 1 to 100
void main()
{
     for(int i=2; i<=100; i=i+2)
     {
          IO.print(i+" ");

          if(i%5==0)
          IO.println();
     }

}
```

```
ForLoopDemo3.java
----------------------
//Find the sum of first 100 natural numbers 1+2+3+4+5.....100
void main()
{
     int sum = 0;

     for(int i=1; i<=100; i++)
     {
          sum = sum + i;
     }
   IO.println("Sum of first 100 natural number :"+sum);
}
```

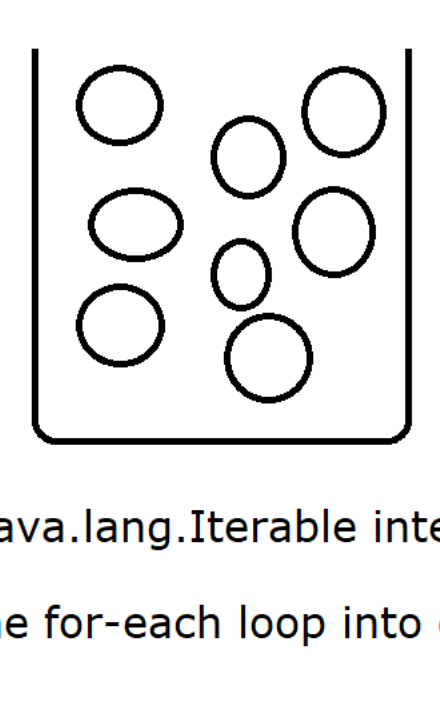
```
ForLoopDemo4.java
----------------------
//use of break
void main()
{
     for(int i=1; i<=10; i++)
     {
          IO.println(i);

      if(i==5)
               break;
     }

}
```

```
ForLoopDemo5.java
----------------------
//use of continue
void main()
{
     for(int i=1; i<=10; i++)
     {
      if(i==5)
               continue;  //Will skip the current iteration of loop

          IO.println(i);
     }

}
```

for each loop in java :
----

* It was introduced from JDK 1.5V and also known as **enhanced for loop.**

* It is used to retrieve the value from the **array OR collection one by one.**



* Here our elemements must implements from java.lang.Iterable interface.

* Our java compiler will automatically convert the for-each loop into ordinary for loop as shown in the diagram

```
ForEachLoop.java
-------------------
void main()
{
  int []values = {10,20,30,40,50};

   //for each loop
   for(int value : values)
     {
          IO.print(value+" ");
     }

}
```

javac

```
ForEachLoop.class
--------------------
void main()
{
  int []values = {10,20,30,40,50};

  for(int i=0; i<values.length; i++)
  {
    IO.println(values[i]);
  }

}
```

```
ForEachLoop1.java
---------------------
//for-each loop

void main()
{
  int []values = {10,20,30,40,50};

   //for each loop
    for(int value : values)
    {
         IO.print(value+" ");
    }

}
```

values is an array variable so, It can hold **multiple values** but value is an **ordinary variable so, can hold only one value at a time**

```
                 10   20  30  40  50
for(int value : values)
{
   IO.println(value); //10  20  30  40 50
}
```

```
ForEachDemo2.java
----------------------
//for-each loop

void main()
{
 String []fruits = {"Apple", "Orange", "Grapes", "Mango"};


  for(String fruit : fruits)
    {
      IO.println(fruit);
    }
}
```